

UNIVERSIDADE FEDERAL DE ITAJUBÁ

Lucas Pires Pivoto – 2024014435

ANALISADOR LÉXICO E SINTÁTICO DA LINGUAGEM SINALÊS

Trabalho apresentado à disciplina ECOM06A – Compiladores,
do curso de Engenharia de Computação da Universidade Federal de Itajubá

Prof^ª. Thatyana de Faria Piola Seraphim

ITAJUBÁ

2026

SUMÁRIO

1 INTRODUÇÃO	3
2 METODOLOGIA	3
2.1 Primeira Etapa: Arquitetura e Regras da Linguagem	3
2.2 Segunda Etapa: Mapeamento Léxico	4
2.3 Terceira Etapa: Formalização	4
2.4 Quarta Etapa: Autômatos	5
3 RESULTADOS	5
3.1 Tabela de Tokens	5
3.2 Expressões Regulares	7
3.3 Regras de Produção (Gramática)	7
3.4 Explicação da Gramática	8
4 AUTÔMATOS	9
5 CONCLUSÃO	15

1 INTRODUÇÃO

Os compiladores são softwares fundamentais para a computação, pois constituem a base para traduzir a linguagem humana em uma linguagem de execução do hardware. Essa tradução permite que haja abstração, sendo convertida uma linguagem de "alto nível" para a execução da máquina em "baixo nível".

No funcionamento dos compiladores, inicia-se com uma análise léxica, na qual o código é dividido em tokens (unidades básicas), que são abordados na linguagem Sinalês como "??", "->", "...", entre outros. Posteriormente, ocorre uma análise sintática do conteúdo escrito para averiguar se a ordem do que foi escrito está correta e faz sentido, seguindo as regras estabelecidas na linguagem.

A proposta do Sinalês baseia-se na restrição de caracteres alfanuméricos aos identificadores, substituindo todos os comandos e estruturas de controle por uma interface puramente simbólica. O objetivo da linguagem é mimetizar visualmente o fluxo de dados e sinais eletrônicos, onde a semântica é ditada pela forma dos caracteres especiais e não por palavras naturais.

O trabalho busca aplicar os conceitos aprendidos na disciplina ECOM06A (Compiladores) ao criar uma linguagem nova e seu analisador léxico e sintático, capaz de processar um programa escrito inteiramente em símbolos, provando a viabilidade de gramáticas não-lexicais.

2 METODOLOGIA

Para garantir a precisão dos analisadores da linguagem, o desenvolvimento foi segmentado em quatro estágios: concepção das regras, mapeamento léxico, formalização gramatical e validação visual por autômatos.

2.1 Primeira Etapa: Arquitetura e Regras da Linguagem

Nesta fase, foram definidas as regras da nova linguagem a ser lida pelo compilador. A linguagem do Sinalês foi baseada em linguagens como Python, C e C++, mas com uma restrição dos caracteres alfanuméricos. Na formulação de suas regras foi estabelecido que:

- É obrigatório o uso de espaços como delimitadores universais (representados por '\n' nessa documentação);
- O uso de ponto e vírgula (;) como terminador de instrução;
- O uso de letras e números apenas para as variáveis, restringindo o restante do código para símbolos;
- Não existe incrementação unária por ++ e --;
- Não existe default no switch;
- Delimitações feitas por chaves.

Em sua maioria, as principais diferenças podem ser vistas nas expressões comuns, que tiveram suas estruturas alteradas para se adaptar ao Sinalês, porém com muitas mantendo a lógica das linguagens progenitoras.

2.2 Segunda Etapa: Mapeamento Léxico

No processo de divisão da linguagem, cada token foi mapeado de forma única, com seus símbolos sendo escolhidos para representar diversos valores, tipos e funções. Nessa etapa, para facilitar a rastreabilidade direta entre o código fonte e o analisador léxico, os símbolos foram classificados e estabeleceu-se a nomenclatura de rastreio como 'T_' + símbolo. Tais definições podem ser vistas na Tabela 1.

2.3 Terceira Etapa: Formalização

Os símbolos isolados não possuem uma semântica computacional; esses precisam ser traduzidos para expressões regulares para possuírem utilidade. Nesse momento, foram construídas as expressões que serão analisadas pelo Analisador Sintático, que definirá se o código é sintaticamente válido e se ele faz sentido ou não. Assim, as regras de produção ditam como os diferentes elementos de comando devem funcionar, como o if, for, while, entre outros. As tabelas referentes a essa construção podem ser visualizadas na seção de Resultados. Para melhor compreensão da linguagem, o sinal '\ ' foi utilizado como forma de representar o espaço, ele não está necessariamente na forma de escrita do Sinalês.

2.4 Quarta Etapa: Autômatos

A última etapa consiste na modelagem dos AFDs (Autômatos Finitos Determinísticos). Através do software JFLAP, mapeou-se o fluxo de estados e transições de cada token, garantindo que o analisador reconheça sequências válidas e aponte erros em cadeias de símbolos inesperadas.

3 RESULTADOS

Nas seções a seguir podem ser vistas as definições e construções do Analisador Léxico e Sintático propostos. A construção desse compilador foi dividida em tabelas, seguindo respectivamente:

- Definição de tokens e palavras reservadas;
- Expressões Regulares;
- Regras de Produção (Gramática);
- Explicação da Gramática.

3.1 Tabela de Tokens

Tabela 1 – Representação e definição dos tokens adotados na linguagem Sinalês

Categoria	Símbolo Original	Símbolo Adotado	Analisador Léxico	Token Original	Token Adotado
Tipos de Dados	int	\$!	Palavra Reservada	T_INT	T_\$!
Tipos de Dados	float	\$@	Palavra Reservada	T_FLOAT	T_\$@
Tipos de Dados	char	\$#	Palavra Reservada	T_CHAR	T_\$#
Variável	letra	letra	Literais de String	T_ID	T_letra
Variável	número	num	Literais Numéricos	T_NUM	T_num
Atribuição	=	->	Operador de Atribuição	T_ATR	T_->
Relacional	==	=	Operador Relacional	T_EQ	T_=
Relacional	!=	!=	Operador Relacional	T_NEQ	T_!=
Relacional	>=	>=	Operador Relacional	T_GEQ	T_>=
Relacional	<=	<=	Operador Relacional	T_LEQ	T_<=
Relacional	>	>	Operador Relacional	T_GT	T_>

Categoria	Símbolo Original	Símbolo Adotado	Analizador Léxico	Token Original	Token Adotado
Relacional	<	<	Operador Relacional	T_LT	T_<
Lógico	&	&&	Operador Lógico	T_AND	T_&&
Lógico			Operador Lógico	T_OR	T_
Lógico	~	~	Operador Lógico	T_NOT	T_~
Lógico	false	[-]	Literal Lógico	T_FALSE	T_-[-]
Lógico	true	[+]	Literal Lógico	T_TRUE	T_[+]
Aritmético	+	+	Operador Aritmético	T_PLUS	T_+
Aritmético	-	-	Operador Aritmético	T_MINUS	T_-
Aritmético	*	*	Operador Aritmético	T_MUL	T_*
Aritmético	/	/	Operador Aritmético	T_DIV	T_/
Aritmético	mod	%	Operador Aritmético	T_MOD	T_%
Aritmético	exp	&	Operador Aritmético	T_E	T_&
Delimitador	;	;	Delimitador	T_DEL	T_;
Delimitador	.	.	Delimitador	T_DOT	T_.
Delimitador	,	,	Delimitador	T_COMMA	T_,
Delimitador	(	(	Delimitador	T_LPAREN	T_(
Delimitador	)	)	Delimitador	T_RPAREN	T_)
Delimitador	{	{	Delimitador	T_LBRACE	T_{
Delimitador	}	}	Delimitador	T_RBRACE	T_}
Delimitador	[	[	Delimitador	T_LBRACKET	T_[
Delimitador	]	]	Delimitador	T_RBRACKET	T_]
Reservada	if	??	Palavra Reservada	T_IF	T_??
Reservada	else	!!	Palavra Reservada	T_ELSE	T_!!
Reservada	for	##	Palavra Reservada	T_FOR	T_##
Reservada	while	@@	Palavra Reservada	T_WHILE	T_@@
Reservada	switch	?!	Palavra Reservada	T_SWITCH	T_?!
Reservada	case	?@	Palavra Reservada	T_CASE	T_?@
Reservada	void	\$()	Palavra Reservada	T_VOID	T_\$()
Reservada	null	...	Palavra Reservada	T_NULL	T_...
Reservada	break	;;	Palavra Reservada	T_BREAK	T_;;
Reservada	return	<<<	Palavra Reservada	T_RETURN	T_<<<
Reservada	define	[]	Palavra Reservada	T_DEFINE	T_[]
Reservada	include	„	Palavra Reservada	T_INCLUDE	T_„
Reservada	cin	<<	Palavra Reservada	T_CIN	T_<<
Reservada	cout	>>	Palavra Reservada	T_COUT	T_>>
Reservada	comentario	//	Palavra Reservada	T_COMMENT	T_//
Reservada	while(true)	{*}	Palavra Reservada	T_FOREVER	T_{*}
Reservada	getDateTime()	@@@	Palavra Reservada	T_DATETIME	T_@@@
Estrutura	main()	[!]	Palavra Reservada	T_MAIN	T_[!]
Comando	Swap	<->	Palavra Reservada	T_SWAP	T_<->

Categoria	Símbolo Original	Símbolo Adotado	Analizador Léxico	Token Original	Token Adotado
Comando	Delay()	.-.	Palavra Reservada	T_DELAY	T_-.-
Controle	restart	<_<	Palavra Reservada	T_RESTART	T_<_<
Controle	Jump(GoTo)	~>	Palavra Reservada	T_GOTO	T_~>
Controle	fatal error	!!!	Palavra Reservada	T_FATERR	T_!!!

Fonte: Elaborado pelo autor (2026).

3.2 Expressões Regulares

Nota: o caractere '\' indica a existência de um espaço ' '.

Tabela 2 – Registro das expressões regulares que fazem parte do Sinalês

Nome	Expressão
num	[0-9]
sinal	[+ -]
veracidade	T_[+] T_-]
exp	T_&
T_num	sinal? num+ (\ num+)? (exp sinal? num+)?
T_letra	[a-zA-Z]+
caracteres	T_num T_letra T_@@@
relacional	T_= T_!= T_>= T_<= T_> T_<
aritmético	T_+ T_- T_* T_/ T_% T_&
logico	T_&& T_ T_~
opRelacional	caracteres (\ relacional \ (caracteres veracidade))+(\ relacional \ (caracteres veracidade))+
opAritmetica	caracteres (\ aritmético \ caracteres) + (\ aritmético \ caracteres)+aritmético \ caracteres)+
opLogica	caracteres (\ logico \ caracteres)+(\ logico \ caracteres)+
condicao	opRelacional (opRelacional \ logico \ condicao)
tipos	T_\$! T_\$@ T_\$# T_\$()
comandos	Condicional Enquanto For Switch Atribuição Entrada Saída Variável Include Define Chamar Função Comentário getDateime() Troca Delay Jump Fatal Error

Fonte: Elaborado pelo autor (2026).

3.3 Regras de Produção (Gramática)

Tabela 3 – Registro das regras de produção da linguagem Sinalês

Nome	Regra
Condicional	T_?? \ T_(\ condicao \ T_) \ T_{ \ comandos+ \ T_} \ (T_!! \ T_{ \ comandos+ \ T_})?
Enquanto	(T_@@ \ T_(\ condicao \ T_)) { * } T_{ \ comandos+ \ T_}

Nome	Regra
For	$T_{##} \setminus T_{(} \setminus \text{Atribuição} \setminus T_{;} \setminus \text{condicao} \setminus T_{;} \setminus \text{opAritmetica} \setminus T_{)} T_{\{ } \setminus \text{comandos} + \setminus T_{\} \setminus \text{comandos} + \setminus T_{;} T_{\}$
Switch	$T_{?!} \setminus T_{(} \setminus T_{\text{letra}} \setminus T_{)} T_{\{ (T_{?@} \setminus \text{caracteres} \setminus T_{;} \setminus \text{comandos} + \setminus T_{;} \setminus T_{;}) + T_{\}$
Atribuição	$T_{\text{letra}} \setminus T_{->} \setminus (\text{caracteres} \mid \text{opAritmetica})$
Entrada (Cin)	$T_{<<} \setminus T_{(} \setminus \text{caracteres} \setminus T_{)} \setminus \text{caracteres} \setminus T_{)}$
Saída (Cout)	$T_{>>} \setminus T_{(} \setminus (\text{caracteres} \mid \text{opAritmetica} \mid \text{veracidade}) \setminus T_{)} T_{(} \setminus (\text{caracteres} \mid \text{opAritmetica} \mid \text{veracidade}) \setminus T_{)}$
Variável	$\text{tipos} \setminus T_{\text{letra}} \setminus T_{;}$
Define	$T_{[]} \setminus T_{\text{letra}} \setminus \text{caracteres}$
Include	$T_{,,} \setminus \text{caracteres}$
Função	$\text{tipos} \setminus T_{\text{letra}} \setminus T_{(} \setminus (\text{tipos} \setminus T_{\text{letra}} \setminus (T_{,,} \setminus \text{tipos} \setminus T_{\text{letra}})^*)? \setminus T_{)} T_{\{ \setminus \text{comandos} + \setminus (T_{<<<} \setminus (\text{caracteres} + \setminus \text{veracidade}))? \setminus T_{\}$
Chamar Função	$T_{\text{letra}} \setminus T_{(} \setminus (\text{caracteres} \setminus (T_{,,} \setminus \text{caracteres})^*)? \setminus T_{)}$
Comentário	$T_{//} \setminus \text{caracteres} +$
Troca	$T_{\text{letra}} \setminus T_{<->} \setminus T_{\text{letra}}$
Main	$T_{[!]} \setminus T_{\{ \setminus \text{comandos} + \setminus (T_{<<<} \setminus (\text{caracteres} \mid \text{veracidade}))? \setminus T_{\}$
Delay	$T_{-.-} \setminus T_{(} \setminus \text{num} + \setminus T_{)}$
Restart	$T_{<_<}$
Jump	$T_{\sim>} \setminus \text{num} +$
Fatal Error	$T_{!!!}$

Fonte: Elaborado pelo autor (2026).

3.4 Explicação da Gramática

Tabela 4 – Finalidade e detalhamento das regras de produção da linguagem Sinalês

Nome	Finalidade Técnica	Detalhamento do Funcionamento
Condicional	Estrutura de decisão (if-else)	$T_{??}$ (if) verifica uma condição em parênteses. Se verdadeira, executa o bloco em chaves; opcionalmente, $T_{!!}$ (else) define o bloco alternativo.
Enquanto	Laço de repetição (while) ou loop infinito; Laço de repetição infinita (loop)	$T_{@@}$ (while) executa um bloco de comandos repetidamente enquanto a condição for avaliada como verdadeira. A estrutura $T_{\{ * \}$ cria um loop infinito onde a condição é sempre verdadeira. Também é presente na estrutura $T_{\{ * \}$, em que a condição sempre é avaliada como verdadeira, o que faz com que o programa funcione em loop.
For	Laço de repetição contado	$T_{##}$ (for) agrupa inicialização (atribuição), condição de parada e incremento (opAritmetica) em um único cabeçalho.
Switch	Seleção múltipla	$T_{?!}$ (switch) avalia uma variável. Cada caso é iniciado por $T_{?@}$ (case) e deve ser encerrado com $T_{;};$ (break).
Atribuição	Definição de valor para variável	Utiliza o operador $T_{->}$ (=) para atribuir o resultado de uma operação ou um valor direto a uma T_{letra} .
Entrada (Cin)	Leitura de dados do usuário	O comando $T_{<<}$ seguido pelo nome da variável armazena o valor inserido externamente.
Saída (Cout)	Exibição de dados na tela	O comando $T_{>>}$ imprime o conteúdo de variáveis ou caracteres literais.

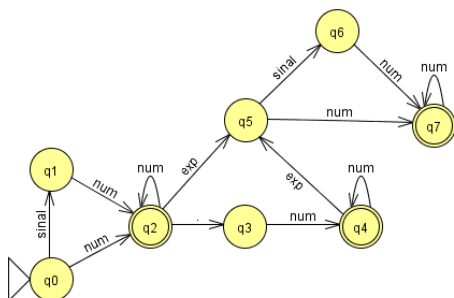
Nome	Finalidade Técnica	Detalhamento do Funcionamento
Variável	Declaração de novo identificador	Define o tipo de dado seguido pelo nome da variável e encerra com T_ _; .
Define	Diretiva de pré-processamento	T_ _[] associa um nome (letra) a um valor constante para substituição global.
Include	Importação de bibliotecas/arquivos	T_ _, , indica ao compilador que deve incluir recursos de um arquivo externo.
Função	Modularização de código	Define um bloco com tipo de retorno e parâmetros. O retorno de valor é feito pelo operador T_ _{<<<} (return) se necessário.
Chamar Função	Recorrer a modularização pré-estabelecida	Chama o bloco modularizado para execução no programa.
Comentário	Documentação interna	Textos precedidos por T_ _{//} são ignorados pelo compilador durante a análise.
Troca (Swap)	Permutação de valores	O operador T_ _{<->} troca diretamente os valores entre duas variáveis distintas.
Main	Ponto de entrada do programa	Identificado por T_ _! . Todo código principal deve estar contido dentro das chaves deste bloco.
Delay	Pausa na execução	O comando T_ ₋ suspende o processamento pelo tempo (num) especificado.
Restart	Reinício do Programa	O comando T_ _{<<} força o programa a reiniciar sua execução, saltando para a linha 0 do programa.
Jump (GoTo)	Desvio de fluxo	T_ _{~>} força o programa a saltar para uma linha ou ponto específico.
Fatal Error	Interrupção crítica	T_ _{!!!!} encerra imediatamente a execução do programa em caso de erro irreversível.

Fonte: Elaborado pelo autor (2026).

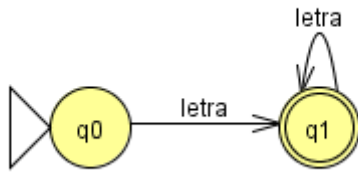
4 AUTÔMATOS

Esta seção destina-se à apresentação dos Autômatos Finitos Determinísticos (AFDs) construídos para cada token da linguagem Sinalês. Os autômatos foram modelados utilizando o software JFLAP e evidenciam o fluxo de estados e transições que o analisador léxico percorre ao reconhecer cada símbolo da linguagem.

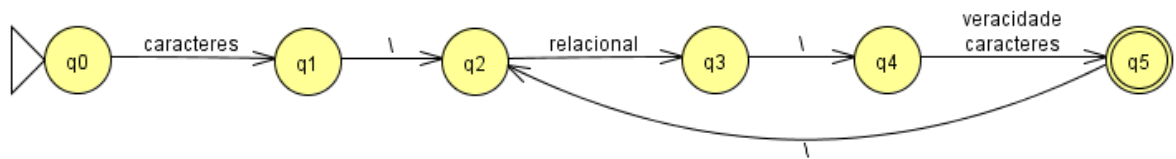
1. T_num



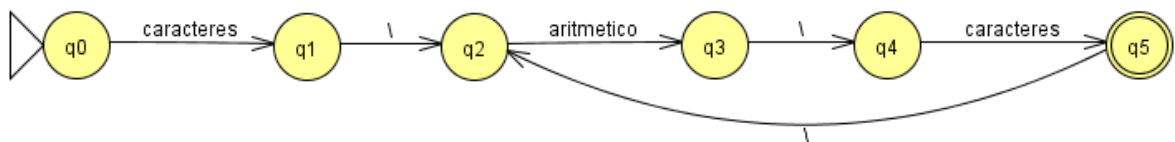
2. T_letra



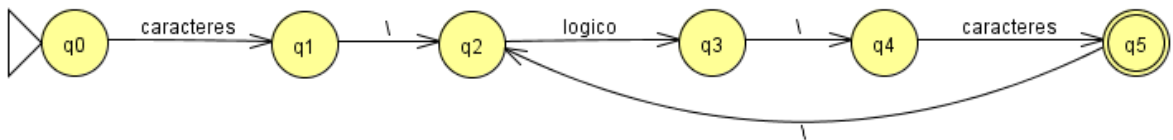
3. OpRelacional



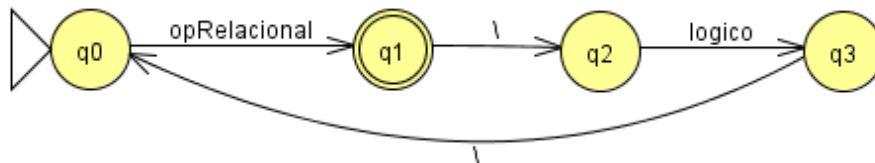
4. OpAritmetica



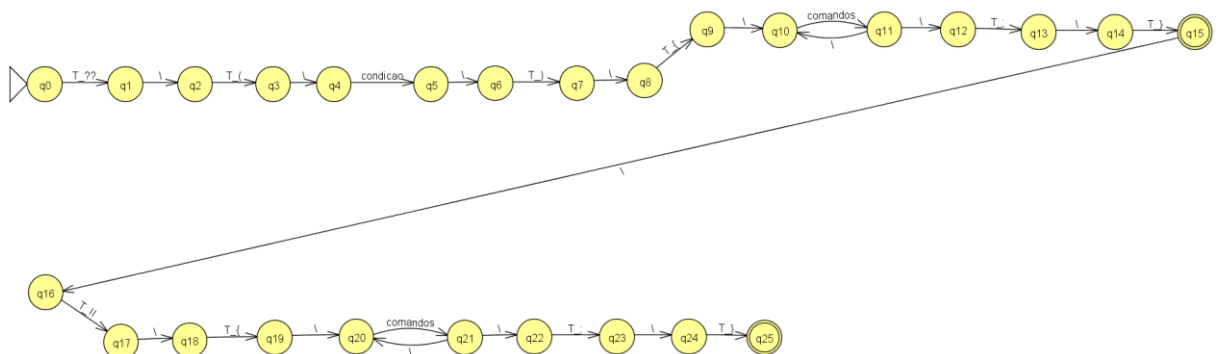
5. OpLogica



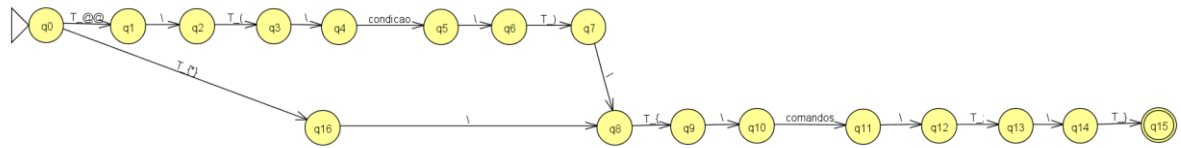
6. Condição



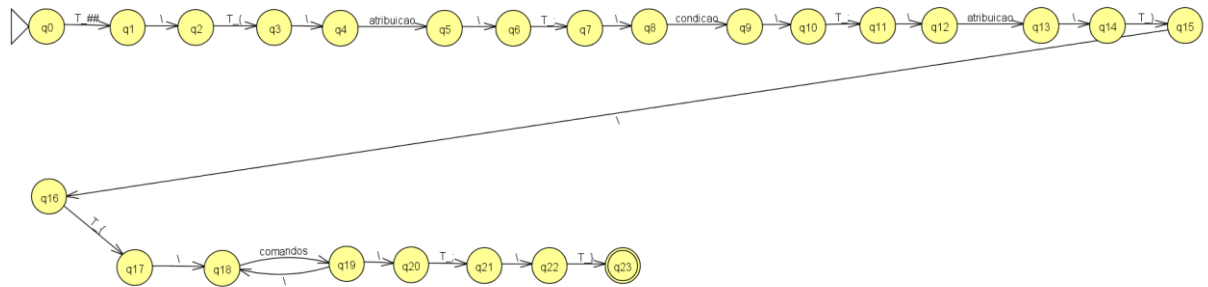
7. Condicional



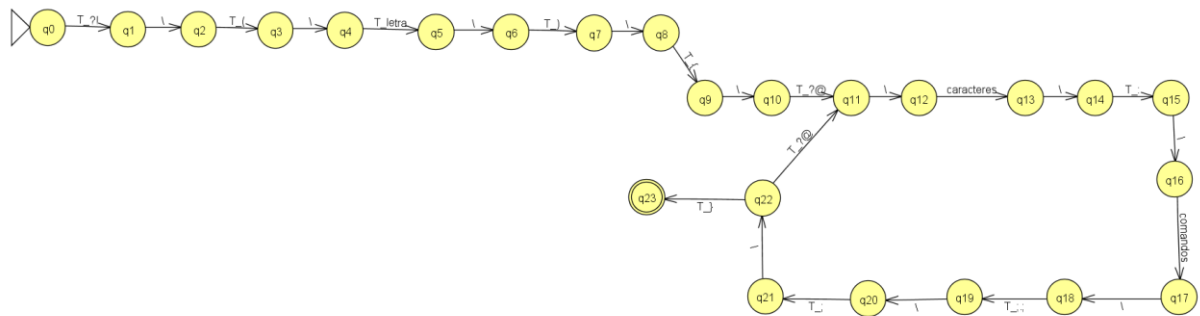
8. Enquanto



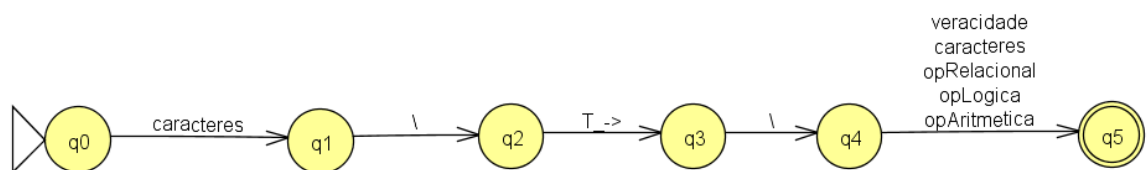
9. For



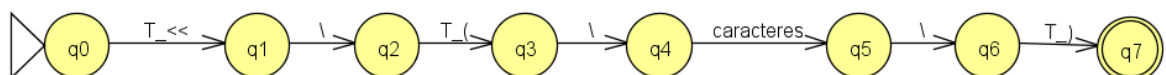
10. Switch



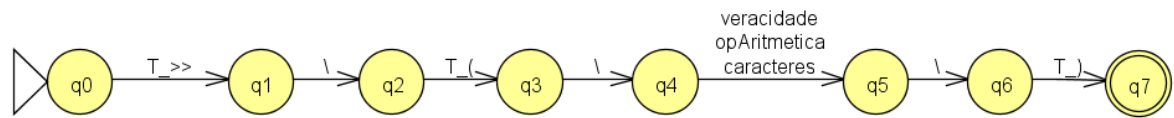
11. Atribuição



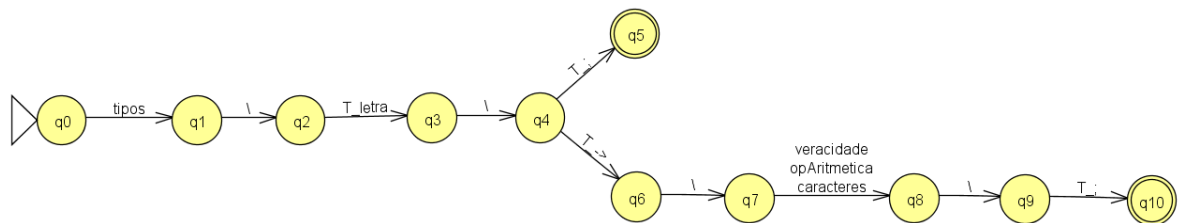
12. Entrada (Cin)



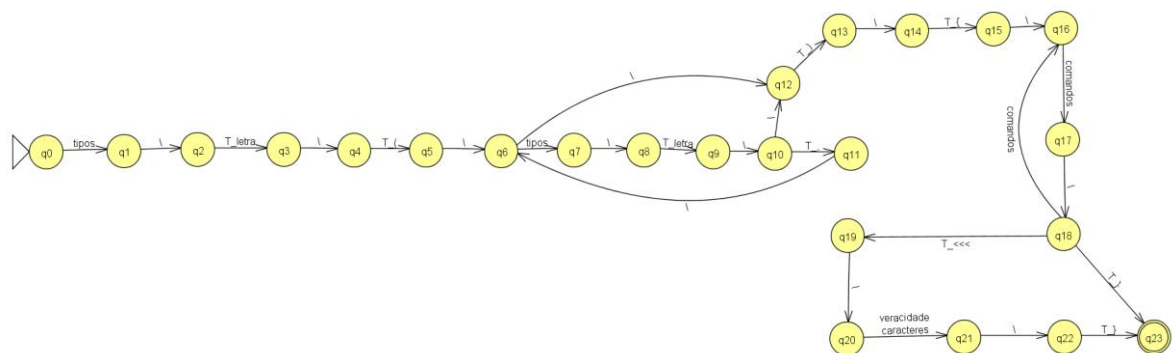
13. Saída (Cout)



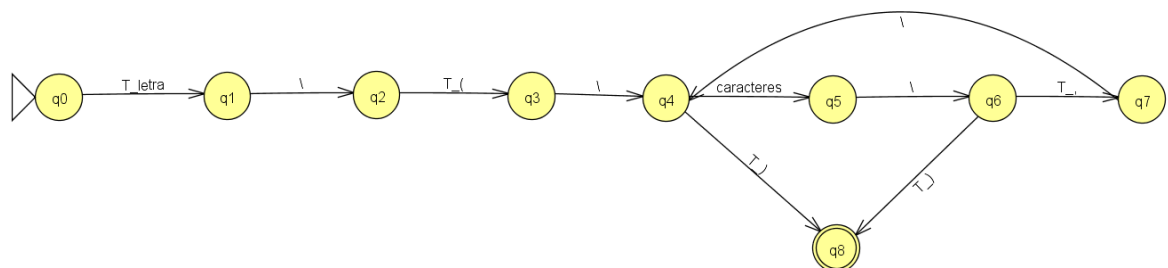
14. Variável



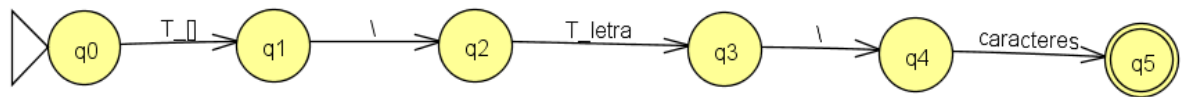
15. Função



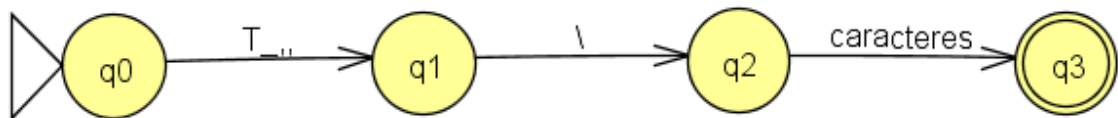
16. Chamar Função



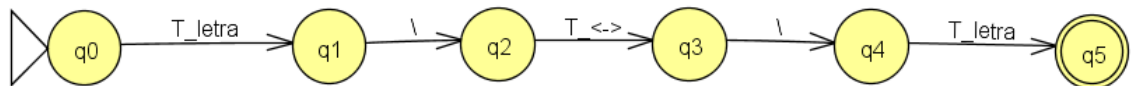
17. Define



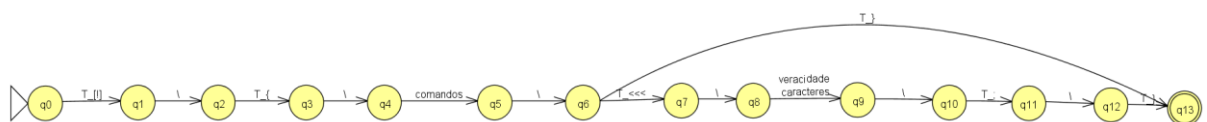
18. Include



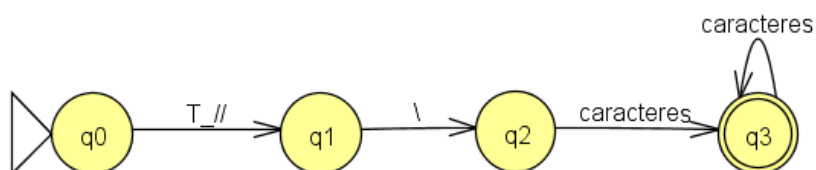
19. Troca



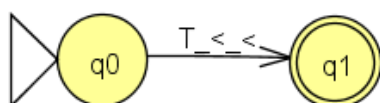
20. Main



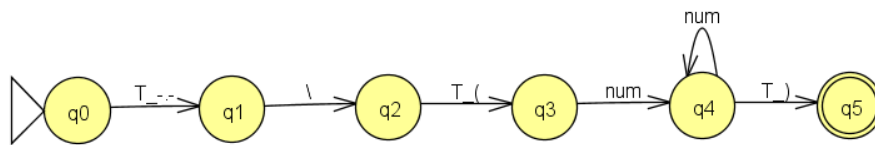
21. Comentário



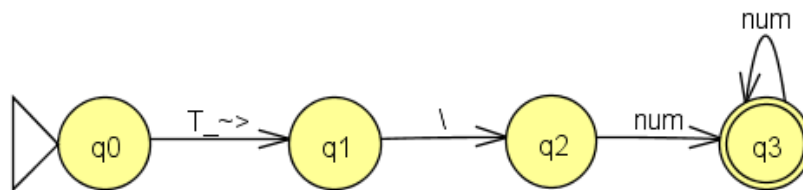
22. Restart



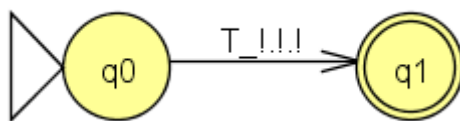
23. Delay



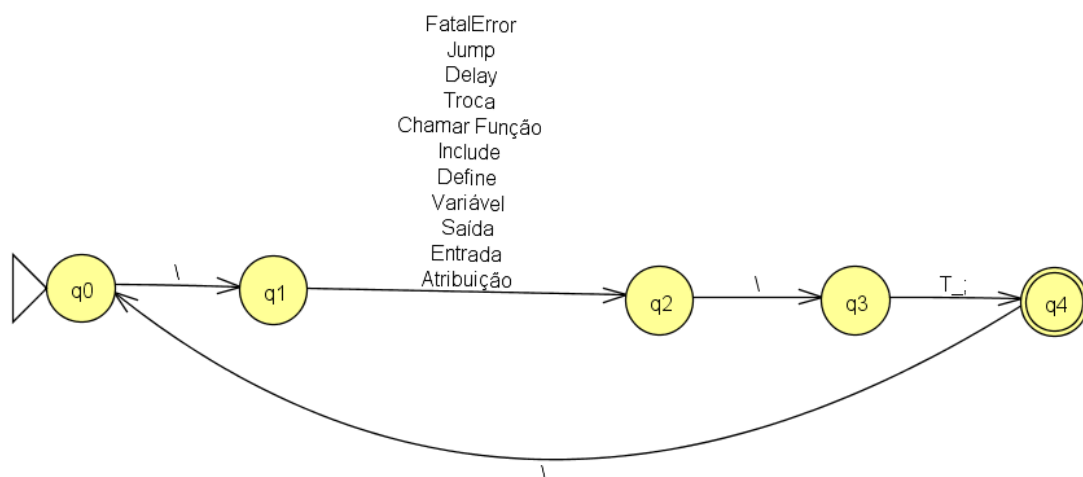
24. Jump



25. Fatal Error



26. Comandos



5 CONCLUSÃO

O presente projeto comprovou que é possível criar uma linguagem de programação composta inteiramente por símbolos especiais. A linguagem Sinalês demonstrou que a substituição de palavras-chave tradicionais por sinais não impede o funcionamento do sistema, sendo perfeitamente possível processá-la com as ferramentas de computação padrão.

A construção dos analisadores léxico e sintático, em conjunto com o desenvolvimento dos autômatos, permitiu aplicar de forma prática os temas estudados na disciplina de Compiladores (ECOM06A). O trabalho percorreu todas as etapas fundamentais, desde a organização inicial dos tokens e expressões regulares até a definição das regras que garantem a estrutura e o funcionamento correto da linguagem.